

250918-INT-PowerTrans-Network

Jingxin Wang

2025-09-18

Objectives of This Attempt

1. Verify if the mean value of non-zero part of methylation data has effect on partial correlations, including Pearson and Spearman Partial Correlation.
2. Explore the implication of the gap between zeros and minimum non-zero values.
3. Do partial correlation change significantly after monotone transformations?

Method:

1. Calculation of Partial Correlations without Transformation, Correlations include
 - Pearson: parametric linear relationship between X and Y
 - Spearman: non-parametric monotone relationship, sensitive to ties
 - Kendall: non-parametric monotone relationship, unsensitive to ties
2. Calculation of Partial Correlations after Transformations
 - Transformations: square root, log, INT
3. Scatter Plot Compare Partial Correlations (before penalty)
 - Take upper triangle of partial correlation matrix of data after any transformation to plot against before transformation
4. Test significance of change in partial correlation

For each transformation, take the absolute difference between calculations with centering with means of non-zero parts and not centering. Plot heatmaps of difference graphs.

1. Loading Data

```
## Load Methylation Data
library(tidyverse)
X.T <- readRDS(file = "../Data/Common_pan_cancer_hyper_bins_adjusted_and_normalized_cnt_in_SE.RDS")
```

2. Apply Transformations on Randomly Selected 5 Rows

```

## Sample 5 DMRs
set.seed(226)
sampDMR <- sample(rownames(X.T), 5)
original.X <- t(X.T[sampDMR, ])

## Box-cox Transformation
source("../R-scripts/INT.R") ## INT maps empirical quantiles to normal(0,1) quantiles

## Boxcox applies monotonic transform
BoxCox <- function(X, lambda) {
  apply(X, MARGIN = 2, FUN = function(x) {
    if (lambda == 0) {
      log(x+1e-6)
    } else {
      (x^lambda - 1)/lambda
    }
  })
}

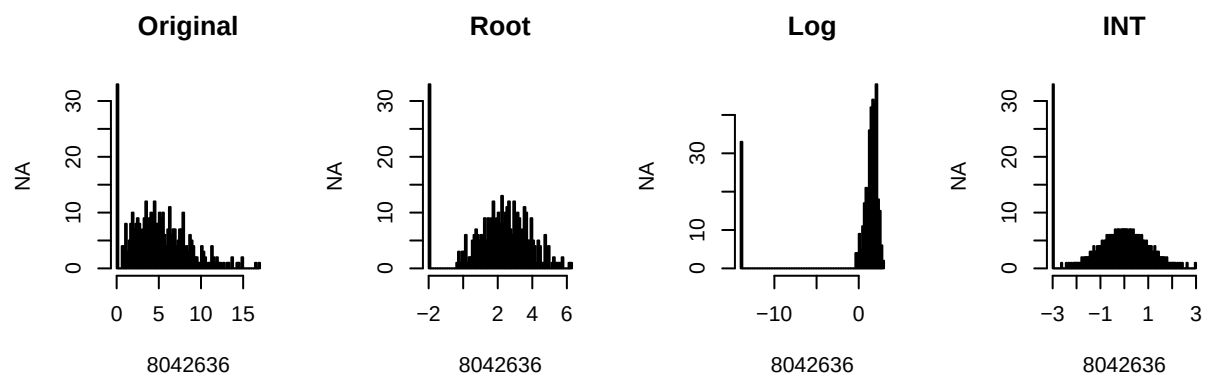
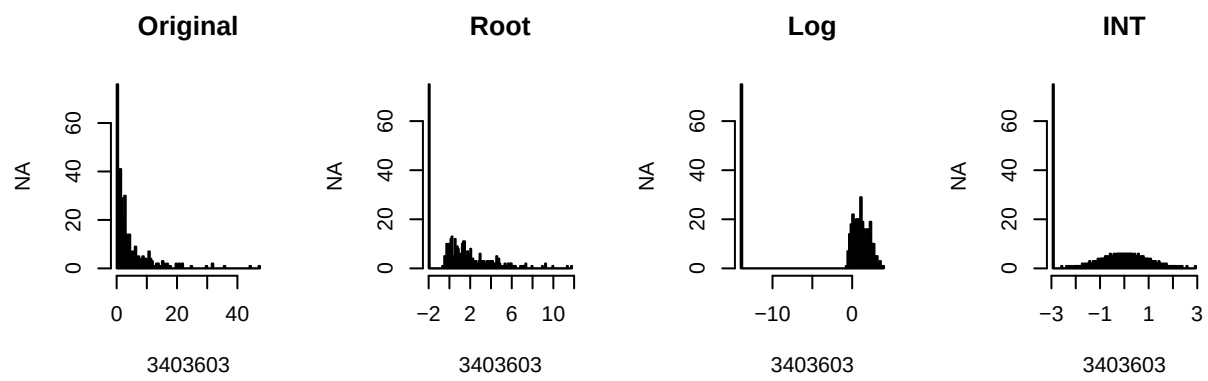
### Root Transform
root.X <- BoxCox(original.X, 0.5)

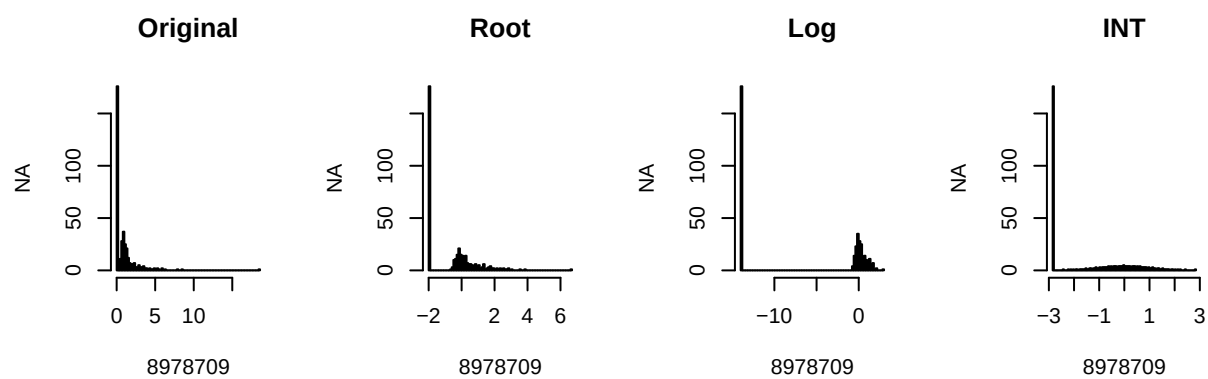
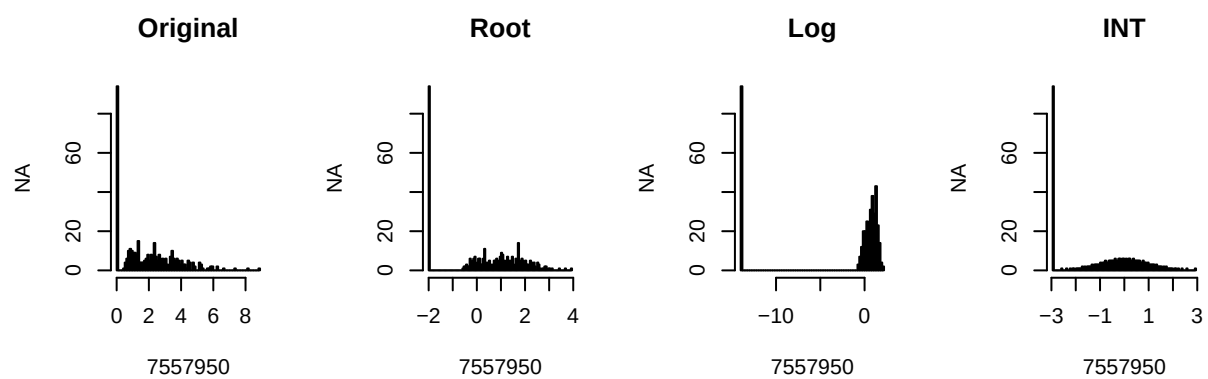
### Log Transform
log.X <- BoxCox(original.X, 0)

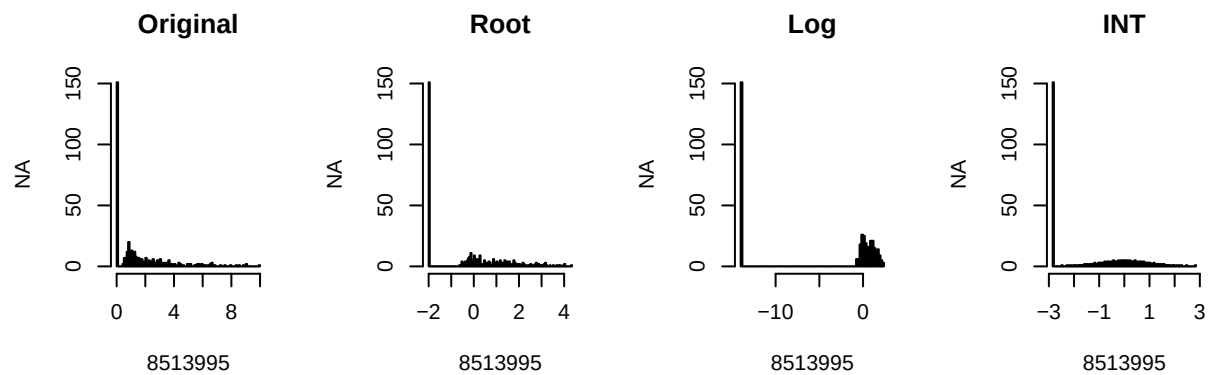
## INT Transformation
INT.X <- INT(original.X, 0.5)

## plot distributions of methylations under different transformations,
## One row per DMR
dmrNames <- colnames(original.X)
par(mfrow = c(2, 4))
for (i in 1:5) {
  hist(original.X[,i], breaks = 100, main = "Original", xlab = dmrNames[i], ylab = 'NA')
  hist(root.X[,i], breaks = 100, main = "Root", xlab = dmrNames[i], ylab = 'NA')
  hist(log.X[,i], breaks = 100, main = "Log", xlab = dmrNames[i], ylab = 'NA')
  hist(INT.X[,i], breaks = 100, main = "INT", xlab = dmrNames[i], ylab = 'NA')
}

```







Observations

1. The gap between zeros and non-zeros widens as the power transformation `lambda` power approaches to 0.
2. Among power transformations, log effectively transforms non-zero part of variables to be symmetrically distributed near 0.
3. After INT, the non-zero part of the distribution is close to normally distributed; however, the distribution is truncated normal.

```
## Evaluate the partial correlations under each scheme before and after transformations
library(ppcor)
```

```
## Loading required package: MASS
```

```
##
```

```
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      select
```

```
allPartialCorr <- function(X) {
  pearsonCorr <- pcor(X, method = "pearson")
  spearmanCorr <- pcor(X, method = "spearman")
  kendallCorr <- pcor(X, method = "kendall")
```

```

  list("pearsonCorr" = pearsonCorr, "spearmanCorr" = spearmanCorr,
       "kendallCorr" = kendallCorr)
}

```

```

originalParRes <- allPartialCorr(original.X)
rootParRes <- allPartialCorr(root.X)
logParRes <- allPartialCorr(log.X)
intParRes <- allPartialCorr(INT.X)

```

```

library(tidyverse)
library(ggplot2)
library(cowplot)

```

```

##
## Attaching package: 'cowplot'

```

```

## The following object is masked from 'package:lubridate':

```

```

##
## stamp

```

```

## Extract partial correlations from upper triangle part of matrices

```

```

extractPar <- function(allParCorr) {
  takeUpperTriangle <- function(parCorr) {
    parMat <- parCorr$estimate
    parVec <- parMat[upper.tri(parMat)]
    as.vector(parMat)
  }
  upperTrigPar <- list()
  for (j in c("pearsonCorr", "spearmanCorr", "kendallCorr")) {
    upperTrigPar[[j]] <- takeUpperTriangle(allParCorr[[j]])
  }
}

```

```

upperTrigPar
}

```

```

## Compare the partial correlations before and after transformations for all types of correlations

```

```

compareParScatter <- function(refPar, expPar) {
  scatterList <- list()
  for (j in c("pearsonCorr", "spearmanCorr", "kendallCorr")) {
    scatterList[[j]] <- ggplot(data = data.frame("Before" = refPar[[j]], "After" = expPar[[j]])) +
      geom_point(aes(x = Before, y = After)) +
      coord_cartesian(xlim = c(0,1), ylim = c(0,1)) +
      geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey") +
      labs(title = j)
  }

  plot_grid(plotlist = scatterList, nrow = 1, ncol = 3)
}

```

```

## Calculation

```

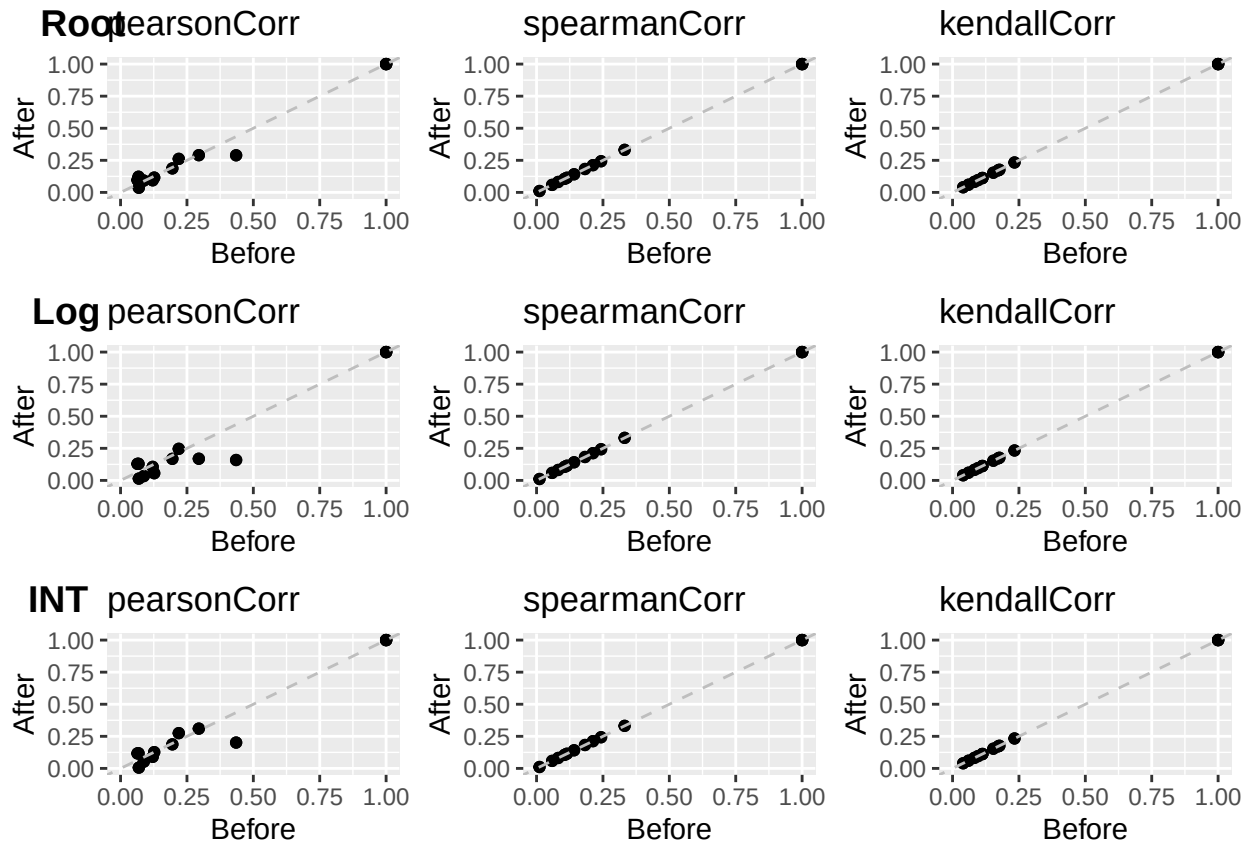
```

originalParEst <- extractPar(originalParRes)
rootParEst <- extractPar(rootParRes)

```

```
logParEst <- extractPar(logParRes)
intParEst <- extractPar(intParRes)

## Plotting
plot_grid(plotlist = list(
  compareParScatter(originalParEst, rootParEst),
  compareParScatter(originalParEst, logParEst),
  compareParScatter(originalParEst, intParEst)
), labels = c("Root", "Log", "INT"),
nrow = 3, ncol = 1)
```



```
ggsave(filename = "../Images/ParCorr/parCorrAfterTrans.png")
```

```
## Saving 6.5 x 4.5 in image
```

Observations:

- Pearson correlations before and after monotone transformations are positively and linearly correlated, with slight differences
- Spearman and Kendall correlations before and after monotone transformations align perfectly

3. Application in Glasso

Graphical Lasso (glasso) estimates the sparse inverse covariance matrix based on the assumption that the data is multivariate normally distributed, and the goal is to help find the penalized version of partial covariance

matrix. It relies on two strong assumptions:

1. The covariance matrix exists. While this exists under Pearson's scheme, it does not under Spearman and Kendall's scheme.
2. It assumes the data is normally distributed, while data in hand could be a mixture of zeros and non-zero random values. Misspecification could make the model invalid.

Additionally, function `pcor()` can calculate partial correlations between any DMR pair, computing the covariance matrix and then inverting it will be extra work. **The hard part is model selection:** when the number of nodes (DMRs) grow to a few hundred, the number of possible edges (interactions between DMRs) is also enormous, we need a solid criterion (p-value for example) to select the edges that matters be able to quantify false discovery rate (FDR) and sensitivity.

We may also wonder if transformed non-zero data can be approximated by normal distributions, this leads us to goodness of fit test of normal distribution.

4. Filter Edges by p-values?

Output of `pcor()` contains both estimate of correlation, test-statistics for 0 correlations, and p-values. The question is 1) What are distributional assumptions of the test statistics? 2) How to interpret the statistics and p-value under 3 correlation scheme?

In general, Given random variables X, Y, Z, and Z has k components to test if the partial correlation is significant, use test statistics

$$T = \rho_{xy.z} \sqrt{\frac{n - k - 2}{1 - \rho_{xy.z}^2}}$$

and the test statistics T follows a $t(n - k - 2)$ distribution, according to (W. Lee, S. Lee and K. Han)[<https://ieeexplore.ieee.org/document/9693250>].

This method only applies to Pearson's correlation and Spearman's correlation (Spearman)[https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient#Determining_significance]. For Kendall tau rank correlation, it has been proven that:

Asymptotic normality – At the $n \rightarrow \infty$ limit, $Z_A = \tau_A / \sqrt{\text{Var}(\tau_A)} = \frac{n_C - n_D}{\sqrt{n(n-1)(2n+5)/18}}$ converges in distribution to the standard normal distribution. See (Kendall)[https://en.wikipedia.org/wiki/Kendall_rank_correlation_coefficient#Hypothesis_test]

Using the original dataset, we will first construct and plot the network using Pearson's partial correlations with edge selected, then we will compare the network with those constructed by Spearman and Kendall correlations. The null hypothesis for testing is $H_0 : \rho_{xy.z} = 0$. With alpha level being 0.05, the null hypothesis will be rejected if the p-value < 0.05 .

```
## We will use ppcor()'s p-value and estimate for now
# install.packages("igraph")
# install.packages("tidygraph")
# install.packages("ggraph")
library(igraph)
```

```
##
## Attaching package: 'igraph'

## The following objects are masked from 'package:lubridate':
##
## %--%, union
```



```
## The following objects are masked from 'package:dplyr':
##
##   as_data_frame, groups, union

## The following objects are masked from 'package:purrr':
##
##   compose, simplify

## The following object is masked from 'package:tidyr':
##
##   crossing

## The following object is masked from 'package:tibble':
##
##   as_data_frame

## The following objects are masked from 'package:stats':
##
##   decompose, spectrum

## The following object is masked from 'package:base':
##
##   union
```

```
library(tidygraph)
```

```
##
## Attaching package: 'tidygraph'

## The following object is masked from 'package:igraph':
##
##   groups

## The following object is masked from 'package:MASS':
##
##   select

## The following object is masked from 'package:stats':
##
##   filter
```

```
library(ggraph)

plot_parcorr_graph <- function(g, corrType) {
  ggraph(g,
    layout = 'linear', circular = TRUE) +
    geom_node_point() +
    geom_edge_arc(aes(colour = weight)) +
    scale_edge_colour_gradient(low="grey",
                              high="black",
                              limits = c(0, 0.4)) +
```

```

    coord_fixed()+
    labs(subtitle = corrType,
         colour = "Pcorr")
}

constructParCorrNetwork <- function(parCorr, alpha, corrType){
  pValMat <- parCorr$p.value
  signifMat <- pValMat < alpha
  estMat <- parCorr$estimate
  networkMat <- estMat * signifMat

  sparseParCorrGraph <-
    graph_from_adjacency_matrix(networkMat,
                                mode = "undirected",
                                weighted = T) %>%

    as_tbl_graph() %>%
    activate(edges) %>%
    filter(from != to)

  plot_parcorr_graph(sparseParCorrGraph, corrType)
}

## Pearson Network
A <- constructParCorrNetwork(originalParRes$pearsonCorr, 0.01, "pearsonCorr")

```

```

## Warning: The 'adjmatrix' argument of 'graph_from_adjacency_matrix()' must be symmetric
## with mode = "undirected" as of igraph 1.6.0.
## i Use mode = "max" to achieve the original behavior.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

```

## Spearman Network
B <- constructParCorrNetwork(originalParRes$spearmanCorr, 0.01, "spearmanCorr")

## Kendall Network
C <- constructParCorrNetwork(originalParRes$kendallCorr, 0.01, "kendallCorr")

plot_grid(plotlist = list(A, B, C), nrow = 1, ncol = 3)

```

```

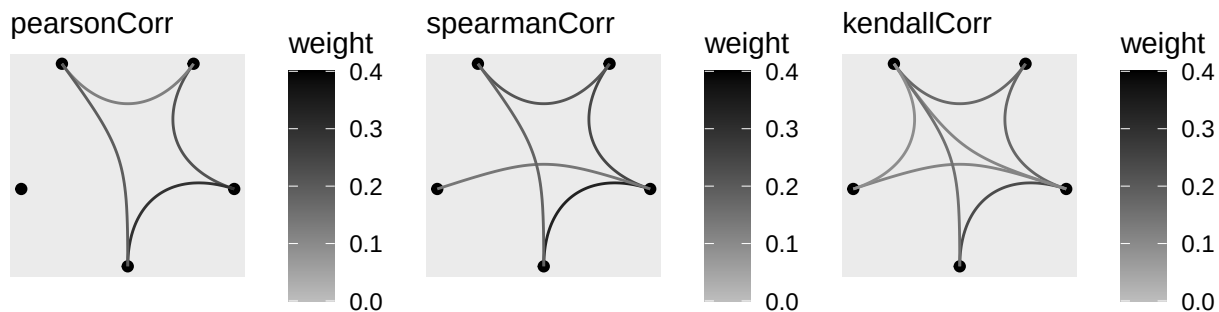
## Ignoring unknown labels:
## * colour : "Pcorr"

```

```

## Ignoring unknown labels:
## * colour : "Pcorr"
## Ignoring unknown labels:
## * colour : "Pcorr"

```



5. Goodness of Fit Test

Model selection by p-values is subject to multiple testing and inflated Type 1 error. We may resume to the For each DMR, the null hypothesis is that non-zero part of the methylation follows a normal distribution $N(\mu, \sigma^2)$; the alternative hypothesis is that it follows some other distribution family. The most widely used test is `shapiro.test()`. If the observed p-value is less than the chosen alpha (test stat < 1), the null hypothesis is rejected. For simplicity, we will test 5 DMRs independently.

Observations: The majority of the test have their p-values < 0.001, meaning that even non-zero part of methylation don't follow normal distributions before and after transformation, which means that modelling based on normal or multivariate normal assumption can be flawed.

```
nonZeroPart <- original.X>0

print("After root transformation")

## [1] "After root transformation"

for (j in 1:5) {
  print(shapiro.test(root.X[nonZeroPart[,j],j]))
}

##
## Shapiro-Wilk normality test
```

```
##
## data: root.X[nonZeroPart[, j], j]
## W = 0.86708, p-value = 1.734e-15
##
##
## Shapiro-Wilk normality test
##
## data: root.X[nonZeroPart[, j], j]
## W = 0.99314, p-value = 0.1171
##
##
## Shapiro-Wilk normality test
##
## data: root.X[nonZeroPart[, j], j]
## W = 0.97916, p-value = 0.0003673
##
##
## Shapiro-Wilk normality test
##
## data: root.X[nonZeroPart[, j], j]
## W = 0.8143, p-value = 8.852e-15
##
##
## Shapiro-Wilk normality test
##
## data: root.X[nonZeroPart[, j], j]
## W = 0.91603, p-value = 5.001e-10
```

```
print("After log transformation")
```

```
## [1] "After log transformation"
```

```
for (j in 1:5) {
  print(shapiro.test(log.X[nonZeroPart[,j],j]))
}
```

```
##
## Shapiro-Wilk normality test
##
## data: log.X[nonZeroPart[, j], j]
## W = 0.97401, p-value = 2.668e-05
##
##
## Shapiro-Wilk normality test
##
## data: log.X[nonZeroPart[, j], j]
## W = 0.97065, p-value = 1.872e-06
##
##
## Shapiro-Wilk normality test
##
## data: log.X[nonZeroPart[, j], j]
## W = 0.97407, p-value = 5.052e-05
```

```
##
##
## Shapiro-Wilk normality test
##
## data:  log.X[nonZeroPart[, j], j]
## W = 0.93737, p-value = 1.207e-07
##
##
## Shapiro-Wilk normality test
##
## data:  log.X[nonZeroPart[, j], j]
## W = 0.96533, p-value = 2.443e-05
```

```
print("After INT transformation")
```

```
## [1] "After INT transformation"
```

```
for (j in 1:5) {
  print(shapiro.test(INT.X[nonZeroPart[,j],j]))
}
```

```
##
## Shapiro-Wilk normality test
##
## data:  INT.X[nonZeroPart[, j], j]
## W = 0.99904, p-value = 0.9999
##
##
## Shapiro-Wilk normality test
##
## data:  INT.X[nonZeroPart[, j], j]
## W = 0.99916, p-value = 0.9999
##
##
## Shapiro-Wilk normality test
##
## data:  INT.X[nonZeroPart[, j], j]
## W = 0.99898, p-value = 0.9999
##
##
## Shapiro-Wilk normality test
##
## data:  INT.X[nonZeroPart[, j], j]
## W = 0.99859, p-value = 0.9998
##
##
## Shapiro-Wilk normality test
##
## data:  INT.X[nonZeroPart[, j], j]
## W = 0.99873, p-value = 0.9998
```

Observations: Normality are violated before and after power transformations, but satisfied after INT.

6. Conclusions

1. Pearson partial correlation is more sensitive to transformation than Spearman and Kendall
2. Normality satisfied after INT transformation.
3. We can experiment Glasso on INT data, and compare the result with network given by Spearman and Kendall
4. Multiple test correction for p-value based edge selection should be tested.

7. Next Step

1. Application of Glasso on zero-inflated data before and after INT transformation
 - Compare the networks before and after transformations
2. Benchmark with Hypothesis Testing
 - Assumptions check for Benjamin-Hotchberg and Benjamin-Yekutielli Test, do they work for correlation of the current type?